

PatriMoi

Architecture technique de la version 1.2 — iOS / React Native

VERSION

1.2 — Juillet 2026

PLATEFORME

iOS (React Native 0.75.4)

BACKEND

Supabase (West EU — Ireland)

STATUT

Production / TestFlight

1 Contexte et portée

PatriMoi est une application iOS de **suivi de patrimoine personnel** destinée aux Marocains résidant au Maroc ou à l'étranger. Elle couvre 8 catégories d'actifs : immobilier, PEA (Bourse de Casablanca), compte-titres, or, assurance-vie, liquidités (comptes bancaires + devises), retraite, et autres.

L'application propose deux modes :

- **Mode démo** — données INIT (~1,5 M DH) pré-remplies, aucun compte requis, persistance locale uniquement (MMKV)
- **Mode connecté** — compte Supabase, sync cloud, snapshots historiques, abonnement PatriMoi+ (49 DH/mois)

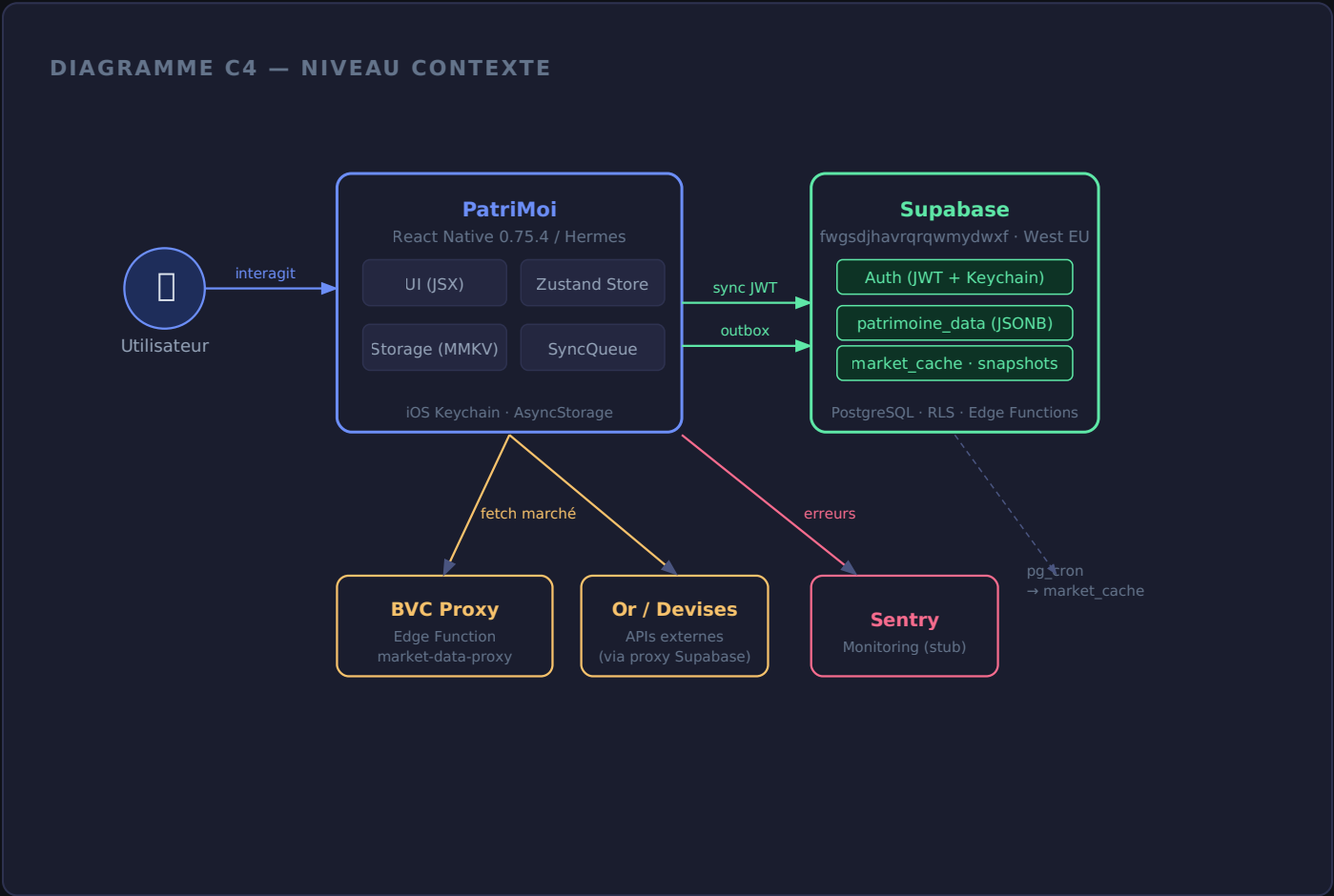
Ce DAT décrit la version 1.2, l'état de l'art post-refactoring des sessions de juillet 2026. Il couvre l'*existant déployé* — pas les évolutions futures.

DIMENSION	VALEUR
Plateforme cible	iOS 16+ (iPhone, iPad)
Framework	React Native 0.75.4 — Old Architecture (Paper renderer)
Moteur JS	Hermes 0.13
Langage principal	JavaScript (.js / .jsx) + TypeScript (.tsx) pour la navigation

Backend	Supabase — projet <code>fwgsdjhavrrqrqwmydwx</code> , région West EU (Ireland)
Stockage local	MMKV (chiffré AES-128) + Keychain iOS + AsyncStorage (legacy)
Distribution	TestFlight (bêta) → App Store

2 Vue d'ensemble de l'architecture

PatriMoi suit une architecture **client-heavy, BaaS-backed** : toute la logique métier tourne dans le client React Native ; le backend (Supabase) assure uniquement la persistance, l'authentification, et le cache des données de marché.



PRINCIPE DIRECTEUR

Le client est la source de vérité pour les données utilisateur. Supabase est la *sauvegarde cloud*, pas un ORM. Toutes les mutations transitent par l'outbox MMKV — Supabase reçoit une copie asynchrone.

3 Infrastructure logicielle

3.1 React Native & moteur Hermes

COMPOSANT	VERSION	RÔLE
React Native	0.75.4	Framework UI mobile
Hermes	0.13	Moteur JS embarqué (AOT compilation, démarrage rapide)
React	18.3	Librairie UI (hooks, reconciler)
Architecture	Old Architecture	Bridge JS↔Native (JSI non activé)
Renderer	Paper	Fabric non activé
Metro	^0.80	Bundler JavaScript
Babel	@react-native/babel-preset	Transformation JSX → CJS

Le choix de rester sur **l'Old Architecture** est intentionnel : la New Architecture (JSI/Fabric) impose une réécriture des native modules et n'apporte pas de gains utilisateur mesurables pour cette app à dominante formulaire. La migration sera envisagée en v2.0.

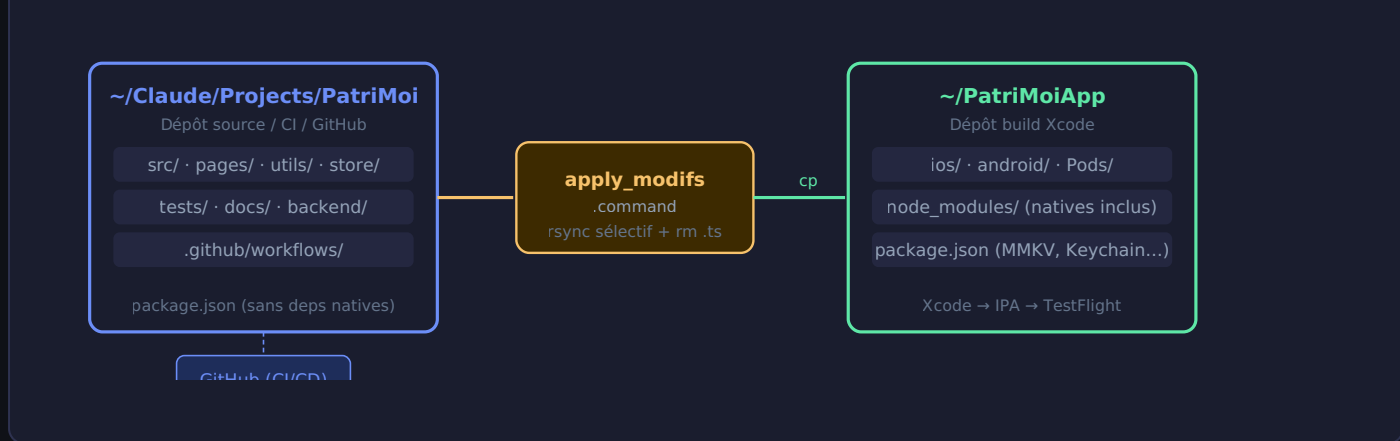
Hermes est le seul moteur supporté — V8 et JavaScriptCore ne font pas partie du pipeline CI. Cela impose deux contraintes :

- `AbortSignal.timeout()` non disponible → `makeTimeoutSignal(ms)` maison avec `AbortController`
- `crypto.getRandomValues()` disponible depuis Hermes 0.13 / RN 0.74 → fallback `Math.random()` inclus dans `_generateKey()` et `_randomSalt()`

3.2 Configuration deux dépôts

La base de code est organisée en **deux répertoires distincts** sur la machine de développement :

SETUP TWO-REPOS — FLUX DE DÉVELOPPEMENT



PatriMoiApp est le projet React Native créé par `react-native init`, avec son Podfile, son projet Xcode et ses modules natifs (`react-native-mmkv`, `react-native-keychain`, etc.) déjà liés et compilés. Le dépôt source **PatriMoi** contient la logique métier, les composants et la CI. Le split est un choix pragmatique : fusionner les deux nécessiterait de recréer la liaison native depuis zéro (CocoaPods, Xcode target, xcconfig). Le script `apply_modifs.command` maintient la synchronisation. À noter : la CI (`ci.yml`) ne compile aucun code natif — elle n'exécute que Jest et `tsc --noEmit` sur `ubuntu-latest` ; le build natif (`release.yml`) tourne sur `macos-15` qui dispose d'Xcode.

Fichiers exclus de la synchronisation (resteront ceux de PatriMoiApp) :

`package.json` · `package-lock.json` · `metro.config.js` · `babel.config.js` · `jest.config.js` · `tsconfig.json` · `index.js` · Podfile · `apply_modifs.command` · `bvc.yml` · `or.yml`

POINT DE VIGILANCE — METRO SHADOWING

Metro résout `.ts` avant `.js`. Si un fichier template `auth.ts` existe dans PatriMoiApp, il *shadow* notre `auth.js` et le module factory peut avorter. `apply_modifs.command` supprime systématiquement les `.ts/.tsx` conflictuels avant chaque déploiement.

4 Couche stockage

PatriMoi utilise une architecture de stockage en **trois niveaux**, chacun avec un rôle précis :

NIVEAU	TECHNOLOGIE	CONTENU	SÉCURITÉ
Niveau 1 (primaire)	MMKV <code>patrimoi_v2</code>	Données patrimoine JSON complètes	AES-128, clé en Keychain
Niveau 1 (clé)	iOS Keychain	Clé AES-128 de MMKV + JWT Supabase + PIN hash	Protégé par des clés dérivées du Secure Enclave, WHEN_UNLOCKED_THIS_DEVICE_ONLY

Niveau 2 (legacy)	AsyncStorage	Copie patrimoine (fallback offline + lazy migration)	Non chiffré
Niveau 3 (outbox)	MMKV <code>patrimoi_v2</code>	Queue mutations, verrou, copie locale la plus récente	AES-128 (même store)

4.1 MMKV chiffré

Deux instances MMKV distinctes sont utilisées :

- `patrimoi_key` (id='patrimoi_key', **non chiffré**) — stockage temporaire de la clé AES-128 *avant* migration Keychain. Vide post-migration.
- `patrimoi_v2` (id='patrimoi_v2', **encryptionKey=<clé AES-128>**) — toutes les données utilisateur + outbox queue.

MMKV est choisi car ses lectures sont **synchrones** (contrairement à AsyncStorage qui impose des `await`), ce qui simplifie le code des reducers Zustand et évite des race conditions lors du chargement des données au démarrage.

4.2 Keychain iOS

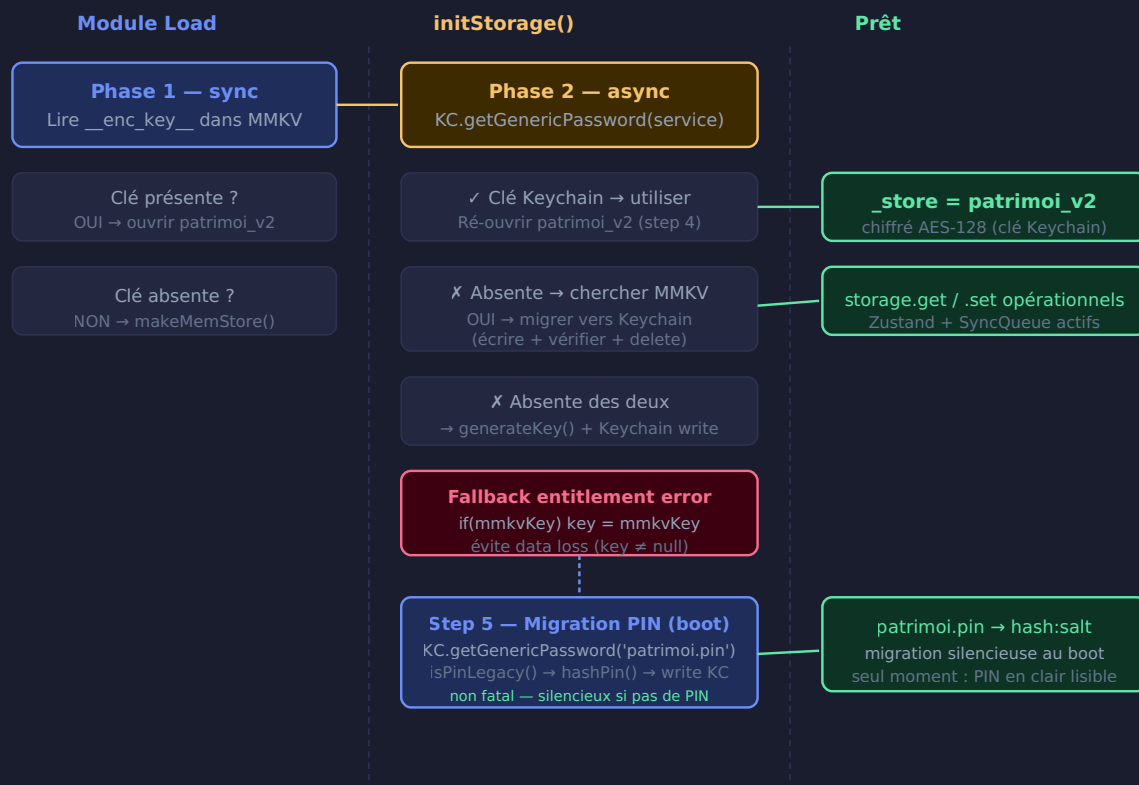
Trois services Keychain sont utilisés, chacun isolé par un service ID distinct :

SERVICE	CONTENU	ACCESSIBLE
<code>patrimoi.mmkv.key</code>	Clé AES-128 pour MMKV patrimoi_v2	WHEN_UNLOCKED_THIS_DEVICE_ONLY
<code>patrimoi.supabase.*</code>	Session JWT Supabase (access + refresh token)	WHEN_UNLOCKED_THIS_DEVICE_ONLY
<code>patrimoi.pin</code>	PIN hashé SHA-256 (<code><hash>:<salt></code>)	WHEN_UNLOCKED_THIS_DEVICE_ONLY

L'attribut `WHEN_UNLOCKED_THIS_DEVICE_ONLY` garantit que la clé est **absente des backups iTunes non chiffrés** et liée à l'appareil physique. La restauration sur un nouvel appareil repart d'une installation fraîche, avec récupération des données depuis Supabase.

4.3 Phases d'initialisation du stockage

SÉQUENCE BOOT — INITIALISATION STOCKAGE (STORAGE.JS)



FIX CRITIQUE V1.2 — PRÉSERVATION DE CLÉ (SCOPE BUG + FALLBACK)

Sur simulateur ou build Debug sans entitlement Keychain Sharing, `KC.setGenericPassword()` lance `errSecMissingEntitlement (-34018)`. Dans le catch du step 2, le fallback `if (mmkvKey) key = mmkvKey` est critique : sans lui, `key` reste `null` → step 3 génère une nouvelle clé → `patrimoi_v2` s'ouvre avec une clé différente → **perte totale des données**.

Bug de scope corrigé (juillet 2026) : `mmkvKey` était déclaré `const` à l'intérieur du bloc `try {}` — une variable `const` / `let` de bloc `try` n'est pas accessible dans le `catch` correspondant. `typeof mmkvKey` renvoyait `'undefined'` (pas de `ReferenceError`, mais condition toujours fausse), rendant le fallback silencieusement inopérant. Corrigé en hoissant la déclaration (`let mmkvKey = null`) avant le `try`.

Sentry : ce cas d'usage émet l'événement `keychain_unavailable_fallback` (filtre entitlement : silencieux sur simulateur, actif sur device sans Keychain Sharing configuré).

STEP 5 — MIGRATION PIN AU BOOT (V1.2.0) — ONE-SHOT

Ajouté dans `initStorage()` après le step 4. Conditionné par le flag `pin_migration_v2` dans `_store` (MMKV `patrimoi_v2`) : si le flag est absent, le step s'exécute — lit `patrimoi.pin` via Keychain, vérifie via `isPinLegacy()`, et si l'ancien format est détecté appelle `hashPin()` pour réécrire en `<sha256>:<salt>`. Le flag est ensuite écrit (`'1'`) que le PIN existe ou non, court-circuitant définitivement le step aux boots suivants.

Pourquoi le flag : un appel `KC.getGenericPassword()` coûte 50–300 ms selon l'état du device. Sans guard, le step 5 ajoutait ce délai à *chaque* démarrage. Avec le flag, le surcoût n'existe qu'au premier boot post-v1.2.0. Le boot est le **seul moment** où la migration est possible sans redemander le PIN à l'utilisateur. `verifyPin()` et `isPinLegacy()` continuent de gérer les deux formats pendant la transition.

5 Composants d'intégration

5.1 Supabase

Supabase fournit quatre services utilisés par PatriMoi :

SERVICE	USAGE PATRIMOI	IMPLÉMENTATION
Auth (GoTrue)	Inscription, connexion, refresh token automatique	<code>supabase.auth.*</code> via <code>keychainStorage</code> adapter
PostgreSQL + RLS	Stockage JSONB patrimoine, profils, snapshots	<code>supabase.from('patrimoine_data')</code> + RPC
Edge Functions	Proxy APIs de marché (BVC, or, devises)	<code>market-data-proxy</code> (Deno)
pg_cron	Mise à jour périodique market_cache	Migration 003 — toutes les 30 min

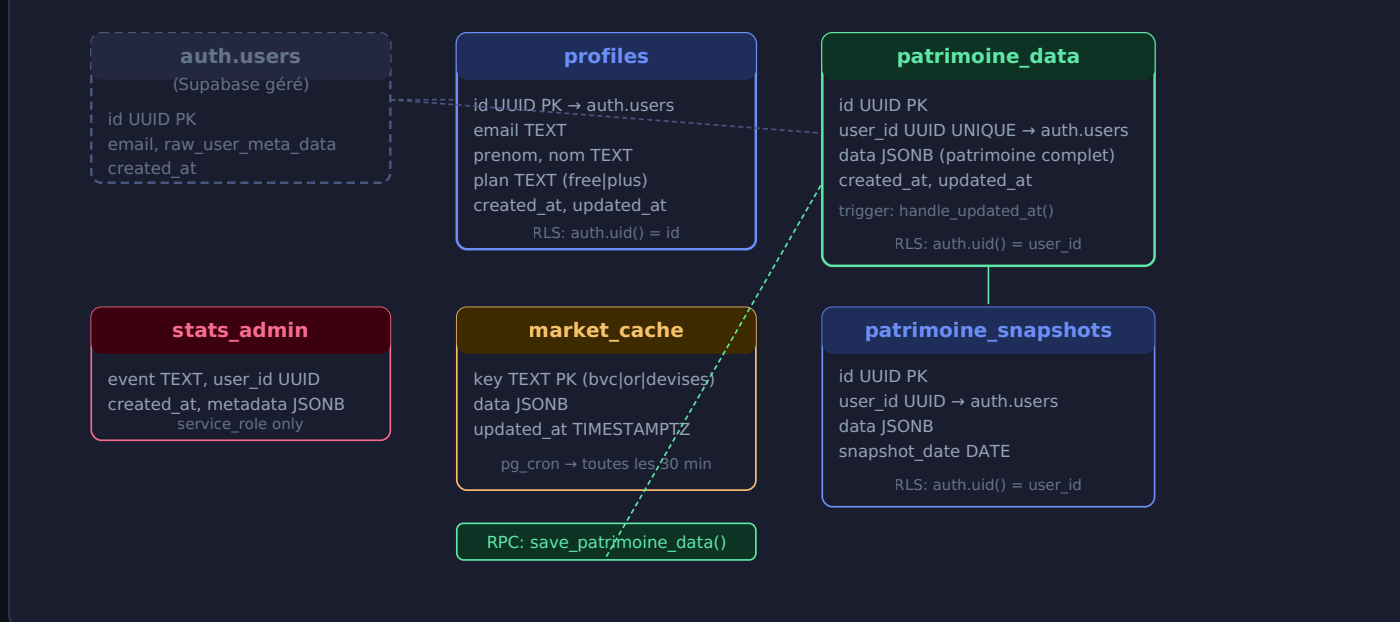
Le client Supabase est initialisé dans `supabase.js` avec le pattern **Zero Static Imports** — tout dans un `try/catch` — et retourne un stub fonctionnel en cas d'échec, pour que l'app reste utilisable en mode démo même si Supabase est indisponible.

KEYCHAINSTORAGE ADAPTER

Le client Supabase utilise `keychainStorage` (défini dans `keychainStorage.js`) comme stockage de session, à la place de l'AsyncStorage par défaut. Cela garantit que les JWT (access token + refresh token) sont stockés dans le Keychain iOS (protégé par des clés dérivées du Secure Enclave), et non dans un fichier texte en clair.

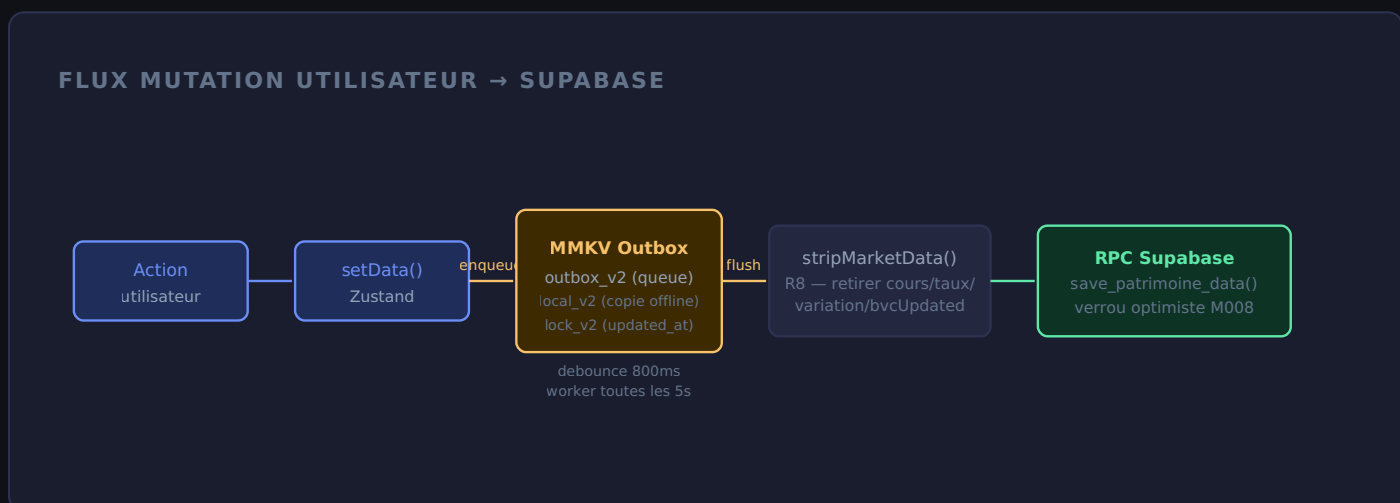
SCHÉMA DE BASE DE DONNÉES

SCHÉMA SUPABASE — TABLES ET RELATIONS



5.2 Outbox pattern (syncQueue.js)

Toutes les mutations utilisateur transitent par un **outbox MMKV** avant d'atteindre Supabase. Ce pattern garantit la durabilité des mutations même en cas de coupure réseau ou de crash de l'application.



Caractéristiques clés de l'outbox :

- **Coalescing** — une seule entrée par `userId` dans la file (last-write-wins), évite les conflits multi-mutations rapides
- **Debounce 800 ms + worker toutes les 5 s** — équilibre réactivité et économie de bande passante
- **Max 5 retries** avec backoff exponentiel (base 1000 ms)
- **Mutations échouées** persistées dans `outbox_failed_v2` (max 20) + alerte Sentry
- **Verrou optimiste (M008)** — RPC `save_patrimoine_data` détecte les conflits multi-appareils et répond `had_conflict: true` sans bloquer l'écriture

5.3 APIs de données de marché

SOURCE	DONNÉE	TRANSPORT	CACHE
Bourse de Casablanca (BVC)	Cours actions marocaines	Edge Function <code>market-data-proxy</code>	MMKV 30 min / stale 24 h
API Or	Prix or (DH/gramme)	Edge Function <code>market-data-proxy</code>	MMKV 30 min
API Devises	Taux EUR/USD/GBP/CHF → MAD	Edge Function <code>market-data-proxy</code>	MMKV 30 min

Ces données sont chargées via `fetchWithRetry()` (3 tentatives, backoff exponentiel). Elles sont stockées dans MMKV mais **jamais envoyées à Supabase** (R8 — strip avant upsert). En cas d'indisponibilité réseau, l'app affiche les dernières valeurs cachées.

5.4 Sentry (monitoring)

Sentry est intégré via un module **stub défensif** : si le package `@sentry/react-native` est absent (CI, build sans DSN), toutes les fonctions deviennent des no-ops silencieux. Deux fonctionnalités de protection :

- **Scrubbing PII** — les montants (`/montant|valeur|total|prix/i`) et les emails sont retirés avant envoi
- **Silence entitlement** — `_isEntitlementError()` filtre les erreurs `errSecMissingEntitlement` (simulateur), évitant de polluer le tableau de bord Sentry

6 Couche données

6.1 Zustand store

L'état global de l'application est géré par un unique store Zustand (`patrimoineStore.js`). Le store est **plat** — pas de slices, pas de Redux Toolkit — ce qui évite les imports circulaires dans le contexte PatriMoiApp.

SLICE LOGIQUE	STATE	SETTERS
Données patrimoine	<code>data</code> (INIT par défaut)	<code>setData(fn)</code> → enqueue sync · <code>setDataRaw(fn)</code> → pas de sync
Navigation	<code>page, sub</code>	<code>setPage(page, sub?)</code>

Auth	<code>user, demoMode, authReady, appReady</code>	<code>setUser, setDemoMode, setAuthReady, setAppReady</code>
Onboarding	<code>onboardingDone</code>	<code>setOnboardingDone</code>
UI / Sync	<code>isRefreshing, bvcStatus, discret, history, isOffline, objectif, trackingStartDate</code>	setters correspondants

SETDATA VS SETDATARAW — DISTINCTION CRITIQUE

`setData(fn)` est le chemin nominal : déclenche `enqueueSync()` si l'utilisateur est connecté et pas en mode démo. `setDataRaw(fn)` est réservé au chargement initial depuis Supabase — utiliser `setData` ici déclencherait une sync immédiate de données déjà sur le serveur, créant des conflits dans l'outbox.

6.2 Schéma de données patrimoine

La structure `data` du store (et de `patrimoine_data.data` en base) contient 8 catégories d'actifs :

CATÉGORIE	CLÉ JSON	STRUCTURE	SPÉCIFICITÉS
Immobilier	<code>immobilier</code>	Array d'objets <code>{id, nom, valeur, ville, type}</code>	Valorisation manuelle
Bourse (PEA)	<code>pea</code>	Array <code>{id, ticker, qte, pru, cours?}</code>	<code>cours</code> stripped avant Supabase (R8)
Compte-titres	<code>ct</code>	<code>{actions[], obligations[], autre}</code>	<code>cours</code> stripped avant Supabase
Or	<code>or</code>	<code>{grammes, prixAchat, prixOr?}</code>	<code>prixOr</code> recalculé au boot
Assurance-vie	<code>assuranceVie</code>	Array <code>{id, nom, valeur, compagnie}</code>	Valorisation manuelle
Liquidités	<code>liquidites</code>	<code>{comptes[], devises[]}</code>	<code>taux/variation</code> devises stripped (R8)
Retraite	<code>retraite</code>	<code>{cimir, rcar, autres[]}</code>	CIMR / RCAR spécifiques Maroc
Autres	<code>autres</code>	Array <code>{id, nom, valeur}</code>	Catch-all

6.3 Migrations de schéma

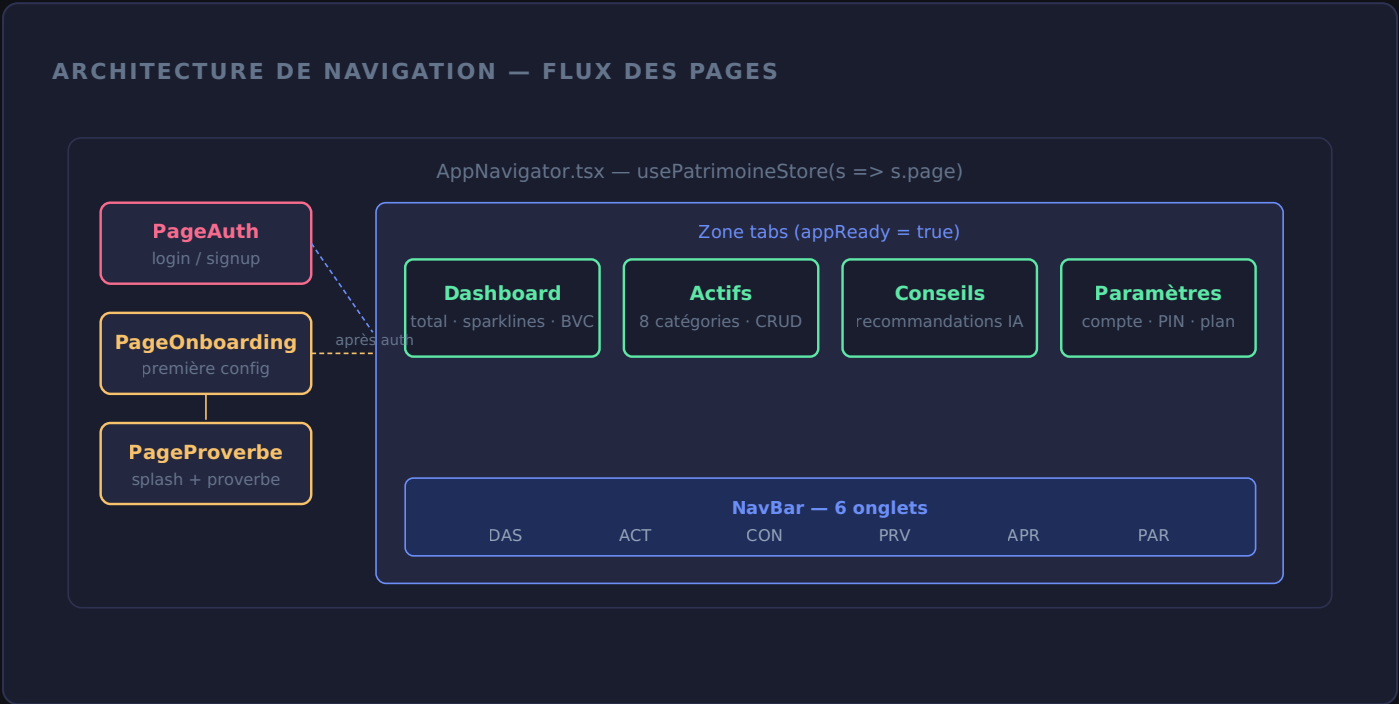
La fonction `migrateData()` (`migrations.js`) gère les évolutions du schéma JSON :

- **Version courante** : `CURRENT_SCHEMA_VERSION = 1`
- **Migration v0→v1** : normalise tous les champs optionnels (ajoute les propriétés manquantes avec valeurs par défaut)
- **Validation structurelle** (`validateData()`) : 8 invariants vérifiés sans Zod (R9 — Zod non disponible dans PatriMoiApp)
- **Fallback INIT** : si la structure est invalide, on repart de zéro plutôt que de crasher

7 Couche IHM (Interface Utilisateur)

7.1 Navigation

PatriMoi utilise un **routeur maison** basé sur Zustand — aucune dépendance à React Navigation ou Expo Router. Le state `page` du store détermine quelle page est rendue par `AppNavigator.tsx`.



Ce choix élimine la complexité de React Navigation (deep linking, stack navigator, etc.) qui n'est pas nécessaire pour une app à navigation tabulaire simple. Le routeur maison est une dizaine de lignes dans `AppNavigator.tsx`.

7.2 Pages

PAGE	FICHER	DESCRIPTION
------	--------	-------------

Proverbe	<code>PageProverbe.jsx</code>	Splash avec proverbe financier aléatoire (10 proverbes marocains)
Auth	<code>PageAuth.jsx</code>	Login / Signup / Reset password
Onboarding	<code>PageOnboarding.jsx</code>	Première configuration (nom, objectif, date de début)
Dashboard	<code>PageDashboard.jsx</code>	Total patrimoine, répartition, sparklines BVC, prix or, taux devises
Actifs	<code>PageActifs.jsx</code>	Liste et édition des 8 catégories d'actifs
Conseils	<code>PageConseils.jsx</code>	Recommandations basées sur la composition du patrimoine
À propos	<code>PageAPropos.jsx</code>	Version, licences, contact
Paramètres	<code>PageParams.jsx</code>	Compte, PIN, plan PatriMoi+, export PDF, mode discret

7.3 Composants partagés

COMPOSANT	FICHER	RÔLE
NavBar	<code>shared.jsx</code>	Barre de navigation inférieure 6 onglets, labels abrégés
SparklineInteractive	<code>shared.jsx</code>	Graphique sparkline avec axes (fix AN_006)
ErrorBoundary	<code>ErrorBoundary.jsx</code>	Capture les erreurs React, affiche écran de fallback

Le layout cible une largeur de **390 pts** (iPhone 14/15 standard). Pas de responsive layout complexe — l'app est iPhone-first.

8 Conventions de développement

8.1 Zero Static Imports pattern

Tous les modules natifs sont chargés via `require()` dans un `try/catch`, jamais via des `import` ES6 statiques. Cette convention est notée dans les commentaires de fichier avec la mention *"IMPORTANT : aucun import statique"*.

```
// ☐ Correct – Zero Static Imports
let KC = null;
```

```

try {
  const K = require('react-native-keychain');
  if (K?.getGenericPassword) KC = K;
} catch {}

// ❌ Incorrect – import statique
import * as Keychain from 'react-native-keychain'; // crashe en CI

```

Raison : avec Babel CJS transform (Old Architecture), les `import` ES6 sont compilés en `require()` qui s'exécutent lors de l'évaluation initiale du module factory. Si un module natif manque (CI, pod non installé), toute la factory avorte et tous ses exports restent `undefined`, propageant l'erreur en cascade. Le pattern `require()` dans `try/catch` isole chaque dépendance et garantit que le module factory se termine toujours.

8.2 Annotations R-series

Les exigences techniques importantes sont annotées dans les commentaires avec un code **R-series** :

CODE	SIGNIFICATION	EXEMPLE D'USAGE
R2	Stockage Keychain (phase 2)	<code>storage.js</code> — init MMKV + Keychain
R3	Verrou optimiste (migration 008)	<code>syncQueue.js</code> → <code>save_patrimoine_data</code> RPC
R5	Sentry monitoring	<code>sentry.js</code> — wrapper défensif
R7	Snapshots historiques	<code>auth.js</code> — <code>loadSnapshots / saveSnapshot</code>
R8	Strip des données de marché	<code>syncQueue.js</code> — <code>stripMarketData()</code>
R9	Validation sans Zod	<code>migrations.js</code> — <code>validateData()</code>

8.3 setData vs setDataRaw — règle absolue

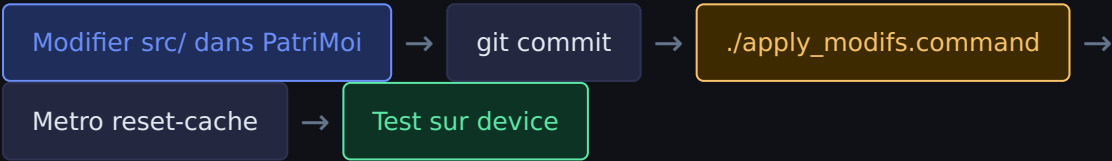
- `setData(fn)` — toute modification des données par l'utilisateur. Déclenche la sync cloud.
- `setDataRaw(fn)` — chargement de données depuis Supabase ou données de marché (cours, taux). Ne déclenche pas la sync (évite les boucles write→sync→write).

8.4 Convention de nommage des fichiers

Les fichiers source PatriMoi sont en `.js` ou `.jsx` (jamais `.ts`/`.tsx` sauf `AppNavigator.tsx`) pour éviter le shadowing Metro. Si un fichier `.ts` du template RN porte le même nom, `apply_modifs.command` le supprime explicitement avant la copie.

Exception — fichiers de test : les fichiers dans `__tests__` peuvent utiliser l'extension `.ts` (convention Jest/TypeScript — ex. `migrations.test.ts`, `store.test.ts`). Metro ne bundle pas le dossier `__tests__`, il n'est donc pas soumis à la règle de shadowing.

8.5 Workflow deux dépôts



9 Sécurité

VECTEUR	MESURE	IMPLÉMENTATION
Données locales	Chiffrement AES-128 MMKV	<code>patrimoi_v2</code> — clé en Keychain
Clé de chiffrement	Keychain iOS (clés dérivées du Secure Enclave)	Keychain <code>WHEN_UNLOCKED_THIS_DEVICE_ONLY</code>
Backup iTunes	Clé absente des backups non chiffrés	Attribut Keychain device-only
PIN utilisateur	SHA-256 + sel aléatoire 128 bits	<code>pinHash.js</code> — format <code><hash>:<salt></code>
Timing attack PIN	Comparaison à temps constant	<code>_constantTimeEqual()</code>
Session Supabase (JWT)	Keychain (pas AsyncStorage)	<code>keychainStorage</code> adapter
Accès données cloud	RLS — isolation stricte par <code>user_id</code>	5 politiques RLS sur toutes les tables
Screenshots	Prévention des captures d'écran	<code>react-native-prevent-screenshot</code>
Données de monitoring	Scrubbing PII avant Sentry	<code>beforeSend</code> — montants + emails filtrés
Clé anon Supabase	Publique par design — sécurité assurée par RLS	Commentaire explicite dans <code>supabase.js</code>

PIN HASHING — ARCHITECTURE DE DÉFENSE EN PROFONDEUR

Un PIN 6 chiffres stocké en clair dans le Keychain serait lisible via un dump Keychain sur appareil jailbreaké. Le module `pinHash.js` implémente SHA-256 pur JS (zéro dépendance native, compatible Hermes) avec sel aléatoire `crypto.getRandomValues()`. Format stocké : `<sha256_hex>:<salt_hex>` (97 caractères).

Migration au boot (v1.2.0) : les devices ayant un PIN en ancien format (texte brut, sans `:`) sont migrés **au premier démarrage post-v1.2.0** dans `initStorage()` step 5. Le step est gardé par un flag `pin_migration_v2` dans MMKV : il ne s'exécute qu'une seule fois, évitant un appel Keychain superflu (50–300 ms) à chaque boot ultérieur.

`isPinLegacy(stored)` détecte le format ancien ; `hashPin(pin)` produit le hash avec sel ; la valeur hachée est réécrite dans `patrimoi.pin`. `verifyPin()` gère les deux formats pendant la période de transition.

10 Tests

FICHIER	COUVERTURE	TYPE
<code>__tests__/calc.test.js</code>	14+ tests — toutes les fonctions de calcul patrimoine	Tests unitaires purs (TNR)
<code>__tests__/migrations.test.ts</code>	Migration v0→v1, CURRENT_SCHEMA_VERSION, validateData()	Tests unitaires
<code>__tests__/phase5.test.js</code>	Migration + mergeHistory + upsertSnapshot + edge cases calc	Tests d'intégration légère
<code>__tests__/store.test.ts</code>	Zustand store actions	Tests unitaires

Stratégie de mock : les modules natifs non testables sont stubés dans `__mocks__` :

- `mmkv.js` — stub MMKV in-memory (`makeMemStore()`)
- `keychain.js` — stub Keychain (get/set en mémoire)
- `prevent-screenshot.js` — no-op
- `async-storage.js` — stub AsyncStorage

Configuration Jest (`jest.config.js`) :

- Preset `react-native`
- Transform TypeScript via `babel-jest` avec `@react-native/babel-preset`
- Coverage minimum : **80 % sur les lignes** (`src/utils/calc.ts`, `migrations.ts`, `store/slices/`)

- Exclusion du boilerplate RN : `App.test.tsx`

11 CI / CD

PIPELINE CI/CD — DE LA PR AU TESTFLIGHT



WORKFLOW	DÉCLENCHEUR	ENVIRONNEMENT	DURÉE ~
<code>ci.yml</code>	Push main/develop + PR → main	ubuntu-latest, Node 18	2-3 min
<code>release.yml</code>	Tag <code>v*.*.*</code>	macos-15, Node 20, Ruby 3.3	20-35 min
<code>bvc.yml</code>	Schedule (30 min) + dispatch	ubuntu-latest	<1 min
<code>or.yml</code>	Schedule (30 min) + dispatch	ubuntu-latest	<1 min

Secrets GitHub requis : `MATCH_PASSWORD`, `MATCH_GIT_URL`, `APP_STORE_CONNECT_KEY_ID`, `APP_STORE_CONNECT_ISSUER_ID`, `APP_STORE_CONNECT_KEY_CONTENT`.

11.2 Configuration Xcode signing

PARAMÈTRE	VALEUR	NOTES
Bundle Identifier	<code>ma.patrimoi.app</code>	⚠ Le template RN génère deux formes dans <code>project.pbxproj</code> — forme littérale <code>et</code> forme template <code>\$(PRODUCT_NAME:rfc1034identifiant)</code> — toutes deux à remplacer
Code Signing Style	Automatic	Xcode gère les profils automatiquement
Team	Compte Apple Developer	À sélectionner dans Signing & Capabilities

Entitlement	<code>keychain-access-</code>	Requis pour <code>react-native-keychain</code> sur device
Keychain	<code>groups</code>	physique — absent = <code>errSecMissingEntitlement (-34018)</code>

PIÈGE COURANT — BUILD SIMULATEUR OK, DEVICE KO

Le simulateur accepte n'importe quel Bundle ID et ignore les entitlements Keychain. Sur device physique deux erreurs bloquent :

1. **No profiles for '...PatriMoiApp'** — le Bundle ID du template RN n'est pas enregistré sur Apple Developer Portal.
2. **Apple ID login -1001** (`NSURLErrorTimedOut`) — timeout réseau lors de la validation du compte. Se résout seul en ouvrant Xcode → Signing & Capabilities → "View Accounts..." : les teams rechargent et l'erreur disparaît.

Procédure de correction du Bundle ID : le `project.pbxproj` contient deux formes distinctes à remplacer par `ma.patrimoi.app` :

- Forme littérale : `org.reactjs.native.example.PatriMoiApp` → corrigée par `fix_device_build.command`
- Forme template : `org.reactjs.native.example.$(PRODUCT_NAME:rfc1034identifier)` → corrigée par `fix_bundle_template2.command`

⚠ La forme template utilise `rfc1034identifier` (pas `rfc822identifier`). Après les deux scripts, fermer et rouvrir `~/PatriMoiApp/ios/PatriMoiApp.xcworkspace` dans Xcode pour recharger le projet.

⚠ **Double app sur le device** : changer le Bundle ID crée une nouvelle entrée iOS. L'ancienne app (`org.reactjs.native.example.PatriMoiApp`) reste installée et doit être supprimée manuellement depuis le device.

12 Argumentaire des choix techniques

12.1 React Native vs alternatives

✓ CHOIX — REACT NATIVE

Partage de logique métier (calculs, migrations, outbox) entre iOS et un futur client Android. Écosystème mature, Hermes rapide. Permet d'itérer sur la logique sans recompiler en Swift.

ALTERNATIVES ÉCARTÉES

Swift natif — plus performant mais pas de partage de code avec Android futur. **Flutter** — excellent mais l'équipe est JS-native, et l'accès Keychain/MMKV est plus limité.

12.2 MMKV vs AsyncStorage

✓ CHOIX — MMKV

Lectures **synchrones** → pas de `await` dans les reducers Zustand. Chiffrement AES-128 natif. 10× plus rapide qu'AsyncStorage sur les benchmarks React Native. API simple (`getString/set/delete`).

ASYNCSOURCE CONSERVÉ

AsyncStorage reste comme couche legacy pour la migration progressive (`smartRead` : MMKV first, fallback AsyncStorage). Sera retiré quand tous les utilisateurs auront migré.

12.3 Keychain pour la clé MMKV

✓ CHOIX — KEYCHAIN

La clé AES-128 dans MMKV était accessible via un backup iTunes non chiffré. Le Keychain avec `WHEN_UNLOCKED_THIS_DEVICE_ONLY` lie la clé à l'appareil physique et l'exclut des backups. Un dump MMKV sans la clé est inutilisable.

MIGRATION GRADUELLE

Les utilisateurs existants (clé dans MMKV) migrent automatiquement à la première ouverture post-v1.2, sans interruption. La Phase 2 async de `initStorage()` gère cette transition.

12.4 Supabase vs Firebase

✓ CHOIX — SUPABASE

PostgreSQL + RLS = sécurité forte et familière. Edge Functions Deno pour le proxy de marché. Région West EU (RGPD). Open-source = pas de vendor lock-in. Coût prévisible à l'usage.

FIREBASE ÉCARTÉ

Firestore est NoSQL — inadapté à un schéma patrimoine évolutif. Les règles de sécurité Firebase sont moins expressives que RLS PostgreSQL pour l'isolation par `user_id`.

12.5 Zustand vs Redux

✓ CHOIX — ZUSTAND

API minimaliste (20 lignes de store), pas de boilerplate action/reducer/selector. Sélecteurs natifs (`usePatrimoineStore(s => s.field)`) sans `connect()` HOC. Bundle size minimal.

REDUX ÉCARTÉ

Redux Toolkit aurait imposé des slices, des middlewares, et une configuration RTK Query — overhead disproportionné pour un store mono-entité. La distinction `setData` / `setDataRaw` est plus lisible en Zustand.

12.6 Navigation maison vs React Navigation

✓ CHOIX — ROUTEUR ZUSTAND

L'app a une navigation tabulaire simple sans deep linking ni stack imbriqué. React

REACT NAVIGATION POUR LA V2

Si des fonctionnalités de deep linking (notifications push → page spécifique) ou un

Navigation ajouterait ~800 Ko de bundle et une complexité inutile (navigateurs imbriqués, refs, gestureHandler).

back-stack natif sont nécessaires, la migration vers React Navigation sera envisagée.

12.7 Outbox pattern vs appel direct Supabase

✓ CHOIX — OUTBOX MMKV

Un appel direct Supabase dans `setData` perdrait les mutations en cas de coupure réseau ou crash. L'outbox garantit la durabilité : même si l'app est tuée entre une modification et la prochaine flush, la mutation sera envoyée au prochain démarrage.

COMPROMIS — LATENCE DE SYNC

La sync peut arriver jusqu'à 800 ms après la modification (debounce). Acceptable pour une app de patrimoine personnel où la cohérence éventuelle est suffisante et la cohérence temps-réel n'est pas requise.

12.8 SHA-256 pur JS vs SubtleCrypto

✓ CHOIX — SHA-256 PUR JS

`SubtleCrypto.digest()` est asynchrone et non disponible sur tous les contextes Hermes 0.13. L'implémentation pur JS (domaine public, Geraint Luff 2012) est synchrone, zéro dépendance, et suffisante pour hacher un PIN 6 chiffres.

LIMITE ACCEPTÉE

SHA-256 pur JS est ~100× plus lent que SubtleCrypto natif, mais pour un usage ponctuel (vérification PIN), cette latence (~5 ms) est imperceptible. Le sel aléatoire protège contre les rainbow tables.